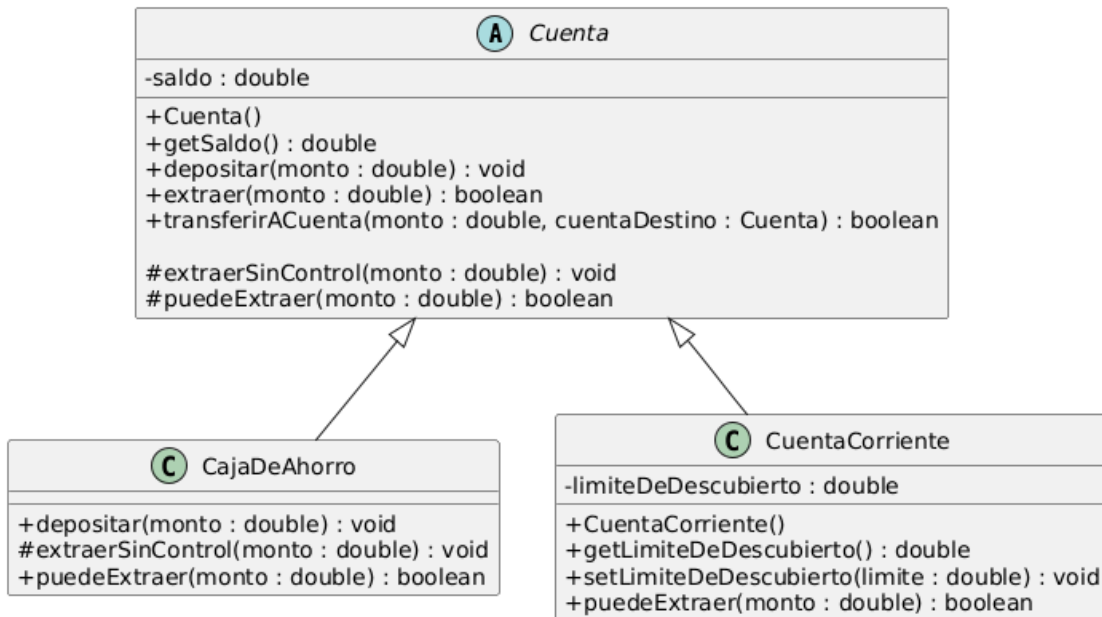


Ejercicio 11 – Cuenta con ganchos



```
public abstract class Cuenta {
    private double saldo;

    public Cuenta() {
        this.saldo = 0;
    }
    public double getSaldo() {
        return this.saldo;
    }
    public void depositar(double monto) {
        this.saldo += monto;
    }
    public boolean extraer(double monto) {
        if(this.puedeExtraer(monto)) {
            this.extraerSinControl(monto);
            return true;
        }
        return false;
    }
    protected void extraerSinControl(double monto) {
        this.saldo -= monto;
    }
    public boolean transferirACuenta(double monto, Cuenta cuentaDestino) {
        if(this.puedeExtraer(monto)) {
            this.extraerSinControl(monto);
            cuentaDestino.depositar(monto);
            return true;
        }
        return false;
    }
    abstract public boolean puedeExtraer(double monto);
}
```

```
public class CajaDeAhorro extends Cuenta{

    public boolean puedeExtraer(double monto) {
        if(this.getSaldo() >= monto * 1.02 ) {
            return true;
        }
        return false;
    }

    protected void extraerSinControl(double monto) {
        super.extraerSinControl(monto*1.02);
    }

    public void depositar(double monto) {
        super.depositar(monto*1.02);
    }
}
```

```
public class CuentaCorriente extends Cuenta{
    private double limiteDeDescubierto;

    public CuentaCorriente() {
        this.limiteDeDescubierto = 0;
    }

    public double getLimiteDeDescubierto() {
        return this.limiteDeDescubierto;
    }

    public void setLimiteDeDescubierto(double limiteDeDescubierto) {
        this.limiteDeDescubierto = limiteDeDescubierto;
    }

    public boolean puedeExtraer(double monto) {
        if(this.getSaldo() + this.getLimiteDeDescubierto() >= monto) {
            return true;
        }
        return false;
    }
}
```

a) Se llama “Cuenta con ganchos” porque la clase Cuenta define el algoritmo general de extracción y transferencia, pero deja ciertos puntos de comportamiento para que las subclases lo completen. Estos métodos son los siguientes:

“puedeExtraer()” y “extraerSinControl()”, estos son llamados los “ganchos” o “hooks”, es decir, puntos donde las subclases pueden personalizar el comportamiento.

b) this es siempre el objeto que recibió el mensaje. Por ejemplo:

```
Cuenta c = new CajaDeAhorro();
```

```
c.extraer(100);
```

```
this.extraer -> objeto CajaDeAhorro
```

Por lo tanto, no se puede afirmar que this sea de la clase Cuenta. Dentro de esos métodos, this es una referencia al objeto real que esta ejecutando la operación, que puede ser una CajaDeAhorro o una CuentaCorriente.

c) La visibilidad de los métodos es depende de a quien queremos que llegue el mismo, por ejemplo el método extraerSinControl() es un método interno del funcionamiento de una Cuenta, no queremos que un Cliente acceda a este método. Sin embargo si necesito que las subclases accedan para redefinirlo por eso es protected, si quiero acceder solo yo y las subclases no, seria private.

d) Si se puede porque el método recibe como parámetro Cuenta cuentaDestino y tanto CajaDeAhorro como CuentaCorriente son subclases de Cuenta.

Esto es posible gracias al polimorfismo: cualquier subtipo de Cuenta puede utilizarse donde se espera una Cuenta.

e) Se declara con la palabra abstract y la clase que lo contiene debe ser abstracta, si es obligatorio implementarlo, salvo que la subclase también sea abstracta.